



North South University
Department of Electrical and Computer Engineering

Report Title:
ISA Design for a 12-bit Single-Cycle CPU

Course code: CSE332

Section: 01

Course Name: Computer Organization C Architecture

Submitted To:

Dr. Tanzilur Rahman

Department of Electrical and Computer Engineering

North South University

Submitted By:

Name	ID
Sumit Hawlader	2413307042
Badrul Hassan Sadman	2413903042
Asef Abdul Awal Khan	2111468042
Hasan Mohammad Abdul Quadir	2312411042

Submission Date: 05/07/2026

1.Introduction

This report proposes an Instruction Set Architecture (ISA) for a new 12-bit single-cycle CPU. The CPU has separate Instruction Memory and Memory. The instruction size is fixed at 12 bits so that instruction fetching and decoding remain simple. The ISA is designed to support arithmetic operations, logical operations, memory access, condition checking, loop execution, jump, and input/output operations.

2. CPU Specification

Feature	Proposed Design
CPU Type	12-bit single-cycle CPU
Instruction Size	Fixed 12 bit
Opcode Size	4 bit
Register File	4 registers
Register Field Size	2 bit
Architecture	Load-store
Instruction Memory	Separate
Memory	Separate
I/O Support	IN, OUT
Main Instruction Formats	R-type, I-type, J-type

3. Architecture Type

The proposed CPU uses load-store architecture. In this architecture, arithmetic and logical operations are performed only on registers. Memory is accessed only by two instructions: LW and SW.

This keeps the datapath and control unit simple for hardware implementation.

```
LW    $s0, 3($zero)    ; $s0 = Memory[3]
LW    $s1, 4($zero)    ; $s1 = Memory[4]
ADD   $t0, $s0, $s1    ; $t0 = $s0 + $s1
SW    $t0, 5($zero)    ; Memory[5] = $t0
OUT   $t0              ; Display result
```

4. Number and Types of Operands

Instruction Type	Number of Operands	Operand Type	Example
R-type	3 registers or registers with shift amount	Register / shift amount	ADD Rd, Rs, Rt / SLL Rd, Rs, shamt
I-type	2 registers + immediate/offset	Register + immediate/offset	ADDI Rt, Rs, imm
J-type	1 address	Address operand	JUMP address
IN	1 register	Destination register	IN Rd
OUT	1 register	Source register	Out Rs

5. Register Table

The CPU has 4 registers. Therefore, each register needs 2 bits because $2^2 = 4$ registers.

R0 is hardwired to zero. Any write operation to R0 is ignored.

Register	Assembly Alias	Binary Code	Purpose
R0	\$zero	00	Constant zero / base register
R1	\$s0	01	Saved/General-purpose register
R2	\$s1	10	Saved/General-purpose register
R3	\$t0	11	Temporary/result register

6. Opcode Size and Instruction Categories

The opcode size is selected as 4 bits. Therefore, the ISA can represent $2^4 = 16$ possible opcodes.

Category	Instructions	Count
Arithmetic	ADD, SUB, ADDI	3
Logic / Shift	AND, OR, SLL, SRL	4
Memory	LW, SW	2
Branch / Jump	BEQ, BNE, JUMP	3
I/O	IN, OUT	2
Reserved	1110, 1111	2

7. Final Instruction and Opcode Table

Opcode	Instruction	Format	Assembly Format (Example)	Execution Operation
0000	ADD	R-type	ADD R1, R2, R3	$R1 = R2 + R3$
0001	SUB	R-type	SUB R1, R2, R3	$R1 = R2 - R3$
0010	AND	R-type	AND R1, R2, R3	$R1 = R2 \text{ AND } R3$
0011	OR	R-type	OR R1, R2, R3	$R1 = R2 \text{ OR } R3$
0100	SLL	R-type	SLL R1, R2, shamt	$R1 = R2 \ll \text{shamt}$
0101	SRL	R-type	SRL R1, R2, shamt	$R1 = R2 \gg \text{shamt}$
0110	ADDI	I-type	ADDI R1, R2, imm	$R1 = R2 + \text{imm}$
0111	LW	I-type	LW R1, offset(R2)	$R1 = \text{Memory}[R2 + \text{offset}]$
1000	SW	I-type	SW R1, offset(R2)	$\text{Memory}[R2 + \text{offset}] = R1$
1001	BEQ	I-type	BEQ R1, R2, offset	If $R1 = R2$, $PC = PC + 1 + \text{offset}$
1010	BNE	I-type	BNE R1, R2, offset	If $R1 \neq R2$, $PC = PC + 1 + \text{offset}$
1011	JUMP	J-type	JUMP address	$PC = \text{jump address}$
1100	IN	R-type	IN R1	$R1 = \text{implicit Input device}$
1101	OUT	R-type	OUT R1	Implicit Output device = R1
1110	Reserved	-	-	Future extension
1111	Reserved	-	-	Future extension

8. Instruction Formats

The ISA mainly uses R-type, I-type, and J-type formats. Shift-type is a variation of R-type, and I/O-type is a special format for external device communication.

8.1 R-Type Format

Used for: ADD, SUB, AND, OR

Opcode	Rs	Rt	Rd	Shamt
4 bit	2	2 bit	2	2 bits

Total =12 bit

8.2 I-Type Format

Used for: ADDI, LW, SW, BEQ, BNE

Opcode	Rs	Rt	Immediate / Offset
4 bit	2	2	4 bit

Total = 4 + 2 + 2 + 4 = 12 bit

For ADDI and branch instructions, the 4-bit immediate/offset is treated as a signed two's complement value. The signed range is -8 to +7.

8.3 J-Type Format

Used for: JUMP

Opcode	Jump Address
4 bit	8 bit

Total = 4 + 8 = 12 bit

9. Addressing Modes

Addressing Mode	Used By	Example	Meaning
Register addressing	ADD, SUB, AND, OR, SLL, SRL	ADD R3, R1, R2	$R3 = R1 + R2$
Immediate addressing	ADDI	ADDI R1, R0, 5	$R1 = R0 + 5$
Base + offset addressing	LW, SW	LW R1, 3(R0)	$R1 = \text{Memory}[R0 + 3]$
PC-relative addressing	BEQ, BNE	BEQ R1, R2, LABEL	If equal, $PC = PC + 1 + \text{offset}$
Direct jump addressing	JUMP	JUMP LOOP	$PC = \text{LOOP address}$

Addressing Mode	Used By	Example	Meaning
Implicit I/O addressing	IN, OUT	IN R1 / OUT R1	Read from fixed input device / write to fixed output device

10. Input/Output Design

The CPU uses dedicated IN and OUT instructions for external communication. The device addresses are implicit and are not encoded inside the instructions. The IN instruction always reads data from the fixed input device, while the OUT instruction always sends data to the fixed seven-segment display.

Instruction	Implicit Device
IN	Keyboard/Input device
OUT	Seven-segment display

```
IN  $s0      ; $s0 = data from the input device
OUT $t0      ; Display the value stored in $t0
```

11. Benchmark Program 1: Simple Arithmetic and Logic

HLL Code

```
input A;
input B;
C = (A + B) OR B;
print(C);
```

Assembly Translation

```
IN $s0          ; $s0 = input A

IN $s1          ; $s1 = input B


ADD $t0, $s0, $s1

OR  $t0, $t0, $s1


OUT $t0         ; Display C
```

12. Benchmark Program 2: Condition Checking

HLL Code

```
if (A == B)
    C = A + B;
else
    C = A - B;
print(C);
```

Assembly Translation

```
BEQ  $s0, $s1, L1      ; If A == B, go to L1


SUB  $t0, $s0, $s1     ; C = A - B

JUMP END               ; Skip TRUE block


L1:

ADD  $t0, $s0, $s1     ; C = A + B
```

```
END:
```

```
OUT    $t0                ; Display C
```

13. Benchmark Program 3: Loop Program

HLL Code

```
sum = 0;
i = N;
while (i != 0) {
    sum = sum + i;
    i = i - 1;
}
print(sum);
```

Assembly Translation

```
ADDI $s1, $zero, 0        ; sum = 0

ADD  $t0, $s0, $zero      ; i = N

LOOP:

BEQ  $t0, $zero, END      ; If i == 0, exit loop

ADD  $s1, $s1, $t0        ; sum = sum + i

ADDI $t0, $t0, -1         ; i = i - 1

JUMP LOOP                 ; Repeat loop
```

```
END:
OUT    $s1                ; Display sum
```

14. Design Tradeoff Discussion

14.1 Number of Registers

The CPU uses 4 registers. Since the instruction size is only 12 bits, 4 registers are selected to keep the register field small. Four registers require only a 2-bit register code. This allows R-type instructions to include one destination register and two source registers within 12 bits.

14.2 Opcode Size

A 4-bit opcode is selected because it provides 16 opcode combinations. This allows most instructions to have separate opcodes and makes instruction decoding easier. The tradeoff is that immediate, offset, and port address fields become smaller.

14.3 Number of Instruction Formats

The ISA mainly uses R-type, I-type, and J-type formats. Shift-type and I/O-type are used as necessary variants. These formats are selected because different instructions need different fields, while the design is still simple enough for hardware implementation.

14.4 Addressing Modes

Simple addressing modes are used: register addressing, immediate addressing, base + offset addressing, PC-relative addressing, direct jump addressing, and I/O port addressing. Complex addressing modes are avoided to keep the datapath simple.

14.5 Load-store vs Accumulator Architecture

Load-store architecture is selected instead of accumulator architecture. In load-store architecture, only LW and SW access memory, while arithmetic and logical operations work on registers. This makes the ISA more general-purpose and easier to translate from high-level programs. Accumulator architecture is simpler, but it depends heavily on a single accumulator register and reduces flexibility.

15. Conclusion

This report presents a 12-bit single-cycle CPU ISA with separate instruction and memory. It uses a 4-bit opcode, 4 registers, fixed 12-bit instructions, and load-store architecture.

The ISA supports arithmetic, logic, memory access, branching, looping, jump, and I/O operations. The IN and OUT instructions allow communication with input devices and seven-segment display, making the design simple and hardware-friendly.